

# Lecture 11: Advanced Event Listeners and DOM Projects

## Event Delegation vs. Multiple Listeners:

- **The Inefficiency:** Applying individual event listeners to multiple child elements (e.g., four separate `<div>` elements) is inefficient and increases the processing load on the program.
- **The Solution (Event Delegation):** A more efficient approach is to attach a single event listener to the parent container.
- Because block-level elements sit inside containers, clicking anywhere inside the parent container triggers the event.

## The Event Object (e) & Target:

- When an event listener is triggered, a callback function is executed and passed an Event Object (commonly named `e`).
- This object stores all information about the event, including the exact location or element that was interacted with.
- The property `e.target` identifies the specific child element that was clicked, allowing for targeted manipulation without needing individual listeners.

```
const parent = document.getElementById("parent");
```

```
parent.addEventListener('click', (e) => {  
  console.log("Parent is clicked");  
  console.log(e.target);
```

```
  e.target.textContent = "I am clicked";  
});
```

## Event Propagation: Bubbling and Capturing:

When elements are nested (e.g., a Child inside a Parent inside a Grandparent), interacting with the innermost element triggers a specific sequence of events.

### 1. Event Bubbling (Default Behaviour):

- If a click occurs on the Child element, the execution starts at the Child, then moves outward to the Parent, and finally to the Grandparent.
- The output order would be: Child is clicked → Parent is clicked → Grand Parent is clicked.
- This outward movement from the target element is called Event Bubbling.

### 2. Event Capturing (Reversing the Order):

- The event execution order can be reversed using an optional third argument in the `.addEventListener()` method.
- By default, this argument is false, placing the execution in the bubbling phase.
- Setting this argument to true places the execution in the Capture Phase.

## The Three Phases of Event Flow:

- **Capture Phase:** Traverses down from the top of the document (HTML → Body) to the target.
- **Target Phase:** Reaches the exact element clicked.
- **Bubbling Phase:** Traverses back up from the target to the top of the document.

```
grandParent.addEventListener("click", () => {  
  console.log("Grand Parent is clicked");  
}, true);
```

## Removing Event Listeners:

- Event listeners should be removed when no longer needed to stop non-productive background execution and free up resources.
- **The Limitation:** Attempting to remove an event listener that uses an anonymous callback function () => {} will fail because the objects have different memory references.
- **The Solution:** A named function must be defined and passed by reference. This allows the function to execute its logic and immediately remove itself so it only runs once.

```
const button = document.querySelector('button');  
  
function handle(){  
  button.textContent = "Clicked";  
  button.removeEventListener('click', handle);  
}  
  
button.addEventListener('click', handle);
```

## Hands-On Projects & Logic Implementation:

### 1. Random Quote Generator:

A button that displays a new random quote from a stored collection on each click.

#### Logic:

- Store quotes in an array.
- Attach a single event listener (`click`) to the button.
- Use `Math.random()` to generate a random index and update the DOM with that quote.

### 2. Background Color Changer:

A set of buttons, each changing the page's background color when clicked.

#### Logic:

- Attach a single event listener on the parent container (Event Delegation) instead of separate listeners per button.
- Method 1: Assign each button an `id` that matches a color name.
- Method 2: Store colors in an array and map them to buttons by index

### 3. Counter:

An increment/decrement counter that tracks a running total.

### Logic:

- Initialize `count = 0`.
- Attach separate `click` listeners to the increment and decrement buttons.
- Update and re-render the count on every click.

**Edge Case:** Prevent the counter from going below zero.

## 4. Calculator:

A form-based calculator triggered on submit.

### Logic:

Forms trigger a page reload by default on submit. Use `e.preventDefault()` to stop this default behavior.

- Attach a `submit` event listener to the form.
- Read input values using `.value`.

**Edge Case:** Input values are received as strings and must be explicitly converted to numbers before performing calculations.

## 5. Word and Character Counter:

A live counter that tracks both word count and character count as the user types.

### Logic:

- Select the `<textarea>` element.
- Attach an `input` event listener to detect changes as the user types.
- Read the current value via `.value` on every keystroke.

**Edge Case:**

- Trim leading/trailing spaces from the text before counting characters.
- Use `.split()` to break the text into an array of words for the word count. This can incorrectly count empty strings when the input is blank, so handle this by checking if the text is empty and setting the word count to 0 directly.
- Use `.filter()` to remove empty entries caused by multiple consecutive spaces, so the word count stays accurate.